Ch8 類別與例外

8-1 類別

- 類別像是一個蛋糕的模子，這個蛋糕的模子可以重複製作出相同的蛋糕，就像類別可以宣告出相同的物件，可以讓程式不斷地被重複利用。
- 在Python 使用class 宣告類別，就可以重複宣告此類別的物件，以下進行類別的實作。

8-1-1　實作類別

- Python 中類別就是使用class 定義類別內的資料與操作的方法
- 方法「__init__」表示宣告類別時會自動執行的方法，第一個參數為self，表示自己，第二個參數為輸入類別的資料，可以在宣告屬於該類別的物件時，同時傳入資料到該物件，傳入的資料可以指定給「self. 變數名稱」，表示該物件有了儲存資料的變數。

8-1-1 實作類別

行號	範例 ( ：ch8\8-1-1- 實作類別 .py)	執行結果
1	class Animal():	
2	def __init__(self, name):	
3	self.name = name	動物
4	a = Animal(' 動物 ')	
5	print(a.name)	

程式解說

- 第1 到3 行：定義類別Animal，宣告方法「__init__」，方法「__init__」輸入參數為self 與name，將name 指定給物件變數self.name。

- 第4行：宣告一個類別Animal的物件，輸入「動物」到類別Animal的初始化方法「__init__」，將此物件貼上物件a 的標籤。
- 第5 行：顯示物件a 的變數name到螢幕上。

8-1-2 繼承

- 可以繼承原有的類別，修改延伸出新的類別，原有的類別被稱為基礎類別(baseclass)或雙親類別(parent class)
- 新的類別被稱為衍生類別(derived class)或子類別(childclass)，這個衍生類別就自動擁有基礎類別的變數與方法。

8-1-2 繼承

- 使用「class 衍生類別(基礎類別)」來定義類別間的繼承關係，衍生類別就繼承了基礎類別
- 在衍生類別中使用「super(). 基礎類別的方法」可以呼叫基礎類別的方法來幫忙，若衍生類別所需要的功能已經在基礎類別定義過了，就可以呼叫基礎類別幫忙，重複利用已經撰寫過的程式碼。

- 當所有衍生類別都需要更改，而且所需功能都相同時，若該功能可以更改基礎類別達成，就直接修改基礎類別，就會影響所有的衍生類別。
- 利用衍生類別與基礎類別的關係達成程式碼的重複利用，減少程式的錯誤，發揮物件導向程式設計的優點。

8-1-2 繼承

行號	範例 (: ch8\8-1-2- 繼承 .py)	執行結果
1	class Animal():	
2	def __init__(self, name):	
3	self.name = name	
4	class Dog(Animal):	
5	def __init__(self, name):	動物
6	super().__init__(' 小狗 '+name)	小狗小白
7	a = Animal(' 動物 ')	
8	d = Dog(' 小白 ')	
9	print(a.name)	
10	print(d.name)	

程式解說

- 第1 到3 行：定義類別Animal，宣告方法「__init__」，方法「__init__」輸入參數為self 與name，將name 儲存到物件變數self.name。

- 第4 到6 行：定義類別Dog 繼承自類別Animal，宣告方法「__init__」，方法「__init__」輸入參數為self 與name，使用「super()」呼叫基礎類別Animal 的方法「__init__」，傳入「小狗」串接name 的字串。

- 第7行：宣告一個類別Animal的物件，輸入「動物」到類別Animal的初始化方法「__init__」，將此物件貼上物件a 的標籤。

- 第8 行：宣告一個類別Dog 的物件，輸入「小白」到類別Dog 的初始化方法「__init__」，將此物件貼上物件d 的標籤。
- 第9 行：顯示物件a 的變數name到螢幕上。
- 第10 行：顯示物件d 的變數name到螢幕上。

8-1-3　覆寫方法

- 衍生類別可以繼承基礎類別的資料與方法，在衍生類別內重新改寫基礎類別的方法，讓衍生類別與基礎類別的相同方法有不同功能，這樣的改寫方法稱作「覆寫」。

- 以下類別Dog 繼承了類別Animal，兩個類別都有方法sound，類別Animal 的方法sound 不執行任何動作，而類別Dog 繼承類別Animal
- 所以有了方法sound，但在類別Dog 中重新覆寫方法sound，讓類別Dog 的方法sound 會印出「汪汪叫」在螢幕上。

8-1-3 覆寫方法

行號	範例 (: ch8\8-1-3- 覆寫函式 .py)	執行結果
1	class Animal():	
2	def __init__(self, name):	
3	self.name = name	
4	def sound(self):	
5	pass	
6	class Dog(Animal):	
7	def __init__(self, name):	小狗小黑
8	super().__init__('小狗 '+name)	汪汪叫
9	def sound(self):	
10	return '汪汪叫 '	
11	d = Dog('小黑 ')	
12	print(d.name)	
13	print(d.sound())	

程式解說

- 第1 到5 行：定義類別Animal，定義方法「__init__」，方法「__init__」輸入參數為self 與name，將name 儲存到物件變數self.name（第2 到3 行）。
- 定義方法sound，使用pass 表示沒有執行任何程式（第4 到5 行）。

- 第6到10行：定義類別Dog繼承自類別Animal，定義方法「__init__」，方法「__init__」輸入參數為self 與name，使用「super()」呼叫基礎類別Animal 的方法「__init__」，傳入「小狗」串接name 的字串（第7 到8 行）。
- 定義方法sound，回傳「汪汪叫」（第9 到10 行）。

- 第11 行：宣告一個類別Dog 的物件，輸入「小黑」到類別Dog 的初始化方法「__init__」，物件d參考到此物件。
- 第12 行：顯示物件d 的變數name 到螢幕上。
- 第13 行：顯示物件d 的方法sound的執行結果到螢幕上。

8-1-4　新增參數的覆寫方法

- 在覆寫方法時，可以新增參數。
- 以下類別Dog 繼承了類別Animal，兩個類別都有方法「__init__」，類別Dog 繼承類別Animal，所以有了方法「__init__」，但在類別Dog 中重新覆寫方法「__init__」時，多出了參數leg，使用self.leg 儲存參數leg。

8-1-4 新增參數的覆寫方法

行號	範例 (: ch8\8-1-4- 新增參數的覆寫函式 .py)	執行結果
1	class Animal():	
2	def __init__(self, name):	
3	self.name = name	
4	def sound(self):	
5	pass	
6	class Dog(Animal):	
7	def __init__(self, name, leg):	小狗小黑 有 4 條腿
8	super().__init__(' 小狗 '+name)	小狗小黑 汪汪叫
9	self.leg = leg	
10	def sound(self):	
11	return ' 汪汪叫 '	
12	d = Dog(' 小黑 ', 4)	
13	print(d.name, ' 有 ', d.leg, ' 條腿 ')	
14	print(d.name, d.sound())	

8-1-4　新增參數的覆寫方法

- 第1 到5 行：定義類別Animal，定義方法「__init__」，方法「__init__」輸入參數為self 與name，將name 儲存到物件變數self.name(第2 到3 行)。
- 定義方法sound，使用pass 表示沒有執行任何程式(第4 到5 行)。

- 第6到11行：定義類別Dog繼承自類別Animal，定義方法「__init__」，方法「__init__」輸入參數為self、name 與leg，使用「super()」呼叫基礎類別Animal 的方法「__init__」，傳入「小狗」串接name的字串，設定self.leg 為leg(第7 到9 行)。
- 定義方法sound，回傳「汪汪叫」(第10 到11 行)。

- 第12 行：宣告一個類別Dog 的物件，輸入「小黑」與「4」到類別Dog 的初始化方法「__init__」，物件d參考到此物件。
- 第13 行：使用函式print 顯示物件d 的變數name，串接「有」，串接物件d 的變數leg，串接「條腿」到螢幕上。

8-1-5　新增方法

- 衍生類別可以繼承基礎類別的資料與方法，在衍生類別內新增基礎類別沒有的方法。
- 以下類別Dog 繼承了類別Animal，兩個類別都有方法「__init__」與方法sound，在衍生類別Dog 新增方法move，而基礎類別Animal 沒有方法move。

8-1-5 新增方法

行號	範例 (: ch8\8-1-5- 新增函式 .py)	執行結果
1	class Animal():	
2	def __init__(self, name):	
3	self.name = name	
4	def sound(self):	
5	pass	
6	class Dog(Animal):	
7	def __init__(self, name):	小狗小黑 汪汪叫 小狗小黑在馬路上行走
8	super().__init__(' 小狗 '+name)	
9	def sound(self):	
10	return ' 汪汪叫 '	
11	def move(self):	
12	print(self.name + ' 在馬路上行走 ')	
13	d = Dog(' 小黑 ')	
14	print(d.name, d.sound())	
15	d.move()	

8-1-5　新增方法

- 第1 到5 行：定義類別Animal，定義方法「__init__」，方法「__init__」輸入參數為self 與name，將name 儲存到物件變數self.name(第2 到3 行)
- 定義方法sound，使用pass 表示沒有執行任何程式(第4 到5 行)。

- 第6到12行：定義類別Dog繼承自類別Animal，定義方法「__init__」，方法「__init__」輸入參數為self 與name，使用「super()」呼叫基礎類別Animal 的方法「__init__」，傳入「小狗」串接name 的字串(第7 到8 行)

- 定義方法sound，回傳「汪汪叫」(第9 到10 行)，定義方法move，印出self.name 串接字串「在馬路上行走」(第11 到12 行)。

- 第13 行：宣告一個類別Dog 的物件，輸入「小黑」到類別Dog 的初始化方法「__init__」，物件d參考到此物件。
- 第14 行：將物件d 的name與呼叫物件d 方法sound的回傳結果顯示在螢幕上。
- 第15 行：呼叫物件d 方法move。

8-1-6　多型(polymorphism)

- 多個類別可以定義相同的方法名稱，而相同方法名稱在不同類別可以定義各自特有的功能
- 經由呼叫物件的方法名稱，傳入不同的物件都定義此相同方法名稱而產生不同的功能，稱作「多型」，在Python 裡這些類別不一定要有繼承關係。

- 以下範例，類別Dog 繼承了類別Animal，兩個類別都有方法who 與方法sound，類別Bird 沒有繼承類別Animal，但有定義相同名稱的方法who 與方法sound

- 所以類別Animal、類別Dog 與類別Bird 就可以實作多型的概念，定義一個函式talk 可以輸入物件，顯示輸入物件的方法who 與方法sound 的回傳結果，經由輸入不同的物件觀察顯示在螢幕上的字串，可以了解多型的概念。

8-1-6 多型(polymorphism)

行號	範例 (: ch8\8-1-6.py)	執行結果
1	class Animal():	
2	def __init__(self, name):	
3	self.name = name	
4	def who(self):	
5	return self.name	
6	def sound(self):	
7	pass	動物 正在 None
8	class Dog(Animal):	小狗小黑 正在 汪汪叫
9	def __init__(self, name):	小鳥小黃 正在 啾啾叫
10	super().__init__('小狗 '+name)	
11	def sound(self):	
12	return '汪汪叫'	
13	class Bird():	
14	def __init__(self, name):	

8-1-6 多型(polymorphism)

```
15          self.name = '小鳥 '+name
16      def who(self):
17          return self.name
18      def sound(self):
19          return ' 啾啾叫 '
20  def talk(obj):
21      print(obj.who(), ' 正在 ', obj.sound())
22  a = Animal(' 動物 ')
23  talk(a)
24  d = Dog(' 小黑 ')
25  talk(d)
26  b = Bird(' 小黃 ')
27  talk(b)
```

8-1-6　多型(polymorphism)

- 第1 到7 行：定義類別Animal，定義方法「__init__」，方法「__init__」輸入參數為self 與name，將name 儲存到物件變數self.name(第2 到3 行)
- 定義方法who，回傳self.name(第4 到5 行)，定義方法sound，使用pass 表示沒有執行任何程式 (第6 到7 行) 。

- 第8到12行：定義類別Dog繼承自類別Animal，定義方法「__init__」，方法「__init__」輸入參數為self 與name，使用「super()」呼叫基礎類別Animal 的方法「__init__」，傳入「小狗」串接name 的字串(第9 到10 行)
- 定義方法sound，回傳「汪汪叫」(第11 到12 行)。

- 第13 到19 行：定義類別Bird，定義方法「__init__」，方法「__init__」輸入參數為self 與name，設定self.name 為「小鳥」串接name 的字串(第14 到15 行)
- 定義方法who，回傳self.name(第16 到17 行)，定義方法sound，回傳「啾啾叫」(第18 到19 行)。

- 第20到21行：定義函式talk，輸入參數obj，印出物件obj的方法who的回傳結果，串接字串「正在」，串接物件obj 的方法sound的回傳結果。
- 第22 行：宣告一個類別Animal 的物件，輸入「動物」到類別Animal 的初始化方法「__init__」，物件a參考到此物件。
- 第23 行：呼叫函式talk，以物件a 為輸入參數。

- 第24 行：宣告一個類別Dog 的物件，輸入「小黑」到類別Dog 的初始化方法「__init__」，物件d參考到此物件。。
- 第25 行：呼叫函式talk，以物件d 為輸入參數。

- 第26 行：宣告一個類別Bird 的物件，輸入「小黃」到類別Bird 的初始化方法「__init__」，物件b參考到此物件。。
- 第27 行：呼叫函式talk，以物件b 為輸入參數。

8-1-7　類別內無法直接存取的變數

- 在類別內的變數，若在變數名稱前加上「__」，該變數無法直接使用「類別物件.__變數名稱」進行存取，達到資料保護的目的，需要在類別內定義方法，回傳「self.__變數名稱」才能存取到該變數。

8-1-7 類別內無法直接存取的變數

行號	範例 (: ch8\8-1-7- 類別內無法直接存取的變數 .py)	執行結果
1	class Animal():	
2	def __init__(self, name):	
3	self.__name = name	
4	def sound(self):	
5	pass	
6	def show_name(self):	
7	return self.__name;	
8	class Dog(Animal):	小狗小黑 有 4 條腿
9	def __init__(self, name, leg):	
10	super().__init__('小狗 '+name)	
11	self.leg = leg	
12	def sound(self):	
13	return '汪汪叫 '	
14	d = Dog('小黑 ', 4)	
15	#print(d.__name,' 有 ', d.leg, ' 條腿 ')	
16	print(d.show_name(), '有 ', d.leg, ' 條腿 ')	

8-1-7　類別內無法直接存取的變數

- 第1 到7 行：定義類別Animal，定義方法「__init__」，方法「__init__」輸入參數為self 與name，將name 儲存到物件變數self.__name(第2 到3 行)
- 定義方法sound，使用pass 表示沒有執行任何程式(第4 到5 行)
- 定義方法show_name 回傳self.__name（ 第6 到7 行 ）。

- 第8到13行：定義類別Dog繼承自類別Animal，定義方法「__init__」，方法「__init__」輸入參數為self、name 與leg，使用「super()」呼叫基礎類別Animal 的方法「__init__」，傳入「小狗」串接name的字串，設定self.leg 為leg(第9 到11 行)。
- 定義方法sound，回傳「汪汪叫」(第12 到13 行)。

- 第14 行：宣告一個類別Dog 的物件，輸入「小黑」與「4」到類別Dog 的初始化方法「__init__」，物件d參考到此物件。
- 第15 行：若將此行的井字號(#) 去除，則會出現AttributeError 錯誤，因為無法直接讀取「d.__name」。

- 第16 行：使用函式print 顯示物件d 的方法show_name 的執行結果，串接「有」，串接物件d 的變數leg，串接「條腿」到螢幕上。

8-1-8　特殊方法(special method)

- 存在於類別內的特殊方法，Python 會讓運算子或內建函式可以與特殊方法自動對應

- 例如判斷兩物件是否相等的運算子「==」會自動與類別內特殊方法「__eq__」
- 所以在類別內重新定義特殊方法「__eq__」，類別中使用運算子「==」的運算就會直接使用特殊方法「__eq__」進行是否相等的判斷

- 以下程式範例就重新改寫特殊方法「__eq__」，判斷兩個物件內的變數name是否相同，決定兩物件是否相等。

8-1-8 特殊方法(special method)

行號	範例 (：ch8\8-1-8.py)	執行結果
1	class Animal():	
2	def __init__(self, name):	
3	self.__name = name	
4	def sound(self):	
5	pass	
6	def show_name(self):	True
7	return self.__name	True
8	def eq(self, other):	False
9	return self.__name == other.show_name()	False
10	def __eq__(self, other):	
11	return self.__name == other.show_name()	
12	class Dog(Animal):	
13	def __init__(self, name, leg):	
14	super().__init__('小狗 '+name)	

8-1-8 特殊方法(special method)

```
15          self.leg = leg
16      def sound(self):
17          return '汪汪叫'
18  d1 = Dog('小黑', 4)
19  d2 = Dog('小黑', 4)
20  print(d1.eq(d2))
21  print(d1 == d2)
22  d3 = Dog('小白', 4)
23  print(d1.eq(d3))
24  print(d1 == d3)
```

8-1-8　特殊方法(special method)

- 第1 到11 行：定義類別Animal，定義方法「__init__」，方法「__init__」輸入參數為self 與name，將name 儲存到物件變數self.name(第2 到3 行)

- 定義方法sound，使用pass 表示沒有執行任何程式（第4 到5 行）

- 定義方法show_name 回傳self.__name(第6 到7 行)，定義方法eq，輸入參數self 與other，回傳self.__name 是否等於物件other 的方法show_name 回傳的字串(第8 到9 行)

- 定義方法「__eq__」，輸入參數self 與other，回傳self.__name 是否等於物件other 的方法show_name 回傳的字串(第10 到11 行)。

- 第12到17行：定義類別Dog繼承自類別Animal，定義方法「__init__」，方法「__init__」輸入參數為self、name 與leg，使用「super()」呼叫基礎類別Animal 的方法「__init__」，傳入「小狗」串接name的字串，設定self.leg 為leg(第13 到15 行)。
- 定義方法sound，回傳「汪汪叫」(第16 到17 行)。

- 第18 行：宣告一個類別Dog 的物件，輸入「小黑」與「4」到類別Dog 的初始化方法「__init__」，物件d1參考到此物件。。
- 第19 行：宣告一個類別Dog 的物件，輸入「小黑」與「4」到類別Dog 的初始化方法「__init__」，物件d2參考到此物件。。

- 第20 行：使用函式print 顯示物件d1 的方法eq 以物件d2 為輸入的運算結果。
- 第21行：使用運算子「==」，判斷物件d1與物件d2是否相等，會自動呼叫方法「__eq__」進行判斷，使用函式print 顯示結果到螢幕上。

- 第 22 行：宣告一個類別 Dog 的物件，輸入「小白」與「4」到類別 Dog 的初始化方法「__init__」，物件d3 參考到此物件。
- 第 23 行：使用函式 print 顯示物件 d1 的方法 eq 以物件 d3 為輸入的運算結果。
- 第24行：使用運算子「==」，判斷物件d1與物件d3是否相等，會自動呼叫方法「__eq__」進行判斷，使用函式print 顯示結果到螢幕上。

8-1-8 特殊方法(special method)

表 8-1 比較運算子與特殊方法

	特殊方法	對應的運算子
比較運算	__eq__(self, other)	self == other
	__ne__(self, other)	self != other
	__gt__(self, other)	self > other
	__ge__(self, other)	self >= other
	__lt__(self, other)	self < other
	__le__(self, other)	self <= other

8-1-8 特殊方法(special method)

表 8-2 算術與邏輯運算子與特殊方法

	特殊方法	對應的運算子
算術與邏輯運算	__add__(self, other)	self + other
	__sub__(self, other)	self - other
	__mul__(self, other)	self * other
	__truediv__(self, other)	self / other
	__floordiv__(self, other)	self // other
	__mod__(self, other)	self % other
	__pow__(self, other)	self ** other
	__lshift__(self, other)	self << other
	__rshift__(self, other)	self >> other
	__and__(self, other)	self & other
	__or__(self, other)	self | other
	__xor__(self, other)	self ^ other

8-1-8　特殊方法(special method)

表 8-3　內建函式與特殊方法

	特殊方法	內建函式
內建函式	__len__(self)	len(self)
	__str__(self)	str(self)
	__repr__(self)	repr(self)

8-1-9 組合(composition)

- 類別與類別之間不全然都是繼承關係，也有可能是類別A 是類別B 的一部分，腳是動物的一部分
- 但腳不是動物，腳無法繼承動物，這時就可以使用組合，在動物類別初始化時，將腳當成參數傳入，讓腳成為動物的一部分。

8-1-9 組合(composition)

行號	範例 (: ch8\8-1-9- 組合 .py)	執行結果
1	class Leg():	
2	def __init__(self, num, look):	
3	self.num = num	
4	self.look = look	
5	class Animal():	
6	def __init__(self, name, leg):	
7	self.__name = name	
8	self.leg = leg	小狗 有 4 隻 短短的 腿
9	def show_name(self):	
10	return self.__name	
11	def show(self):	
12	print(self.show_name(),' 有 ', self.leg.num, ' 隻 ', self.leg.look, ' 腿 ')	
13	leg = Leg(4, ' 短短的 ')	
14	a = Animal(' 小狗 ', leg)	
15	a.show()	

程式解說

- 第1 到4 行：定義類別Leg，定義方法「__init__」，方法「__init__」輸入參數為self、num 與look，將num 儲存到物件變數self.num，將look 儲存到物件變數self.look。

- 第5 到12 行：定義類別Animal，定義方法「__init__」，方法「__init__」輸入參數為self、name 與leg，將name 儲存到物件變數self.__name，將leg 儲存到物件變數self.leg(第6 到8 行)

- 定義方法show_name 回傳self.__name(第9 到10 行)
- 定義方法show，顯示方法self.show_name的結果，串接「有」，串接self.leg.num，串接「隻」，串接self.leg.look，串接「腿」(第11 到12 行)。

- 第13 行：宣告一個類別Leg 的物件，輸入「4」與「短短的」到類別Leg 的初始化方法「__init__」，變數leg參考到此物件。
- 第14 行：宣告一個類別Animal 的物件，輸入「小狗」與leg到類別Animal 的初始化方法「__init__」，變數a參考到此物件。
- 第15 行：在螢幕上顯示物件a 的方法show 的結果。

8-1-10　類別方法

- 目前所介紹的類別內方法，都屬於實例方法(instance method)，此方法的第一個參數都是self。

- 類別方法(class method) 作用對象為類別，會影響整個類別，也會影響類別所產生的物件，類別方法的第一個參數通常取名為cls，需在類別中方法的前一行使用裝飾器「@classmethod」，這樣的方法稱作「類別方法」。

8-1-10 類別方法

行號	範例 (: ch8\8-1-10- 類別方法 .py)	執行結果
1	class Animal():	
2	count = 0	
3	def __init__(self):	
4	Animal.count += 1	
5	def kill(self):	
6	Animal.count -= 1	
7	@classmethod	
8	def show_count(cls):	現在有 1 隻動物
9	print(' 現在有 ',cls.count,' 隻動物 ')	現在有 2 隻動物
10	a = Animal()	現在有 3 隻動物
11	Animal.show_count()	現在有 2 隻動物
12	b = Animal()	
13	Animal.show_count()	
14	c = Animal()	
15	Animal.show_count()	
16	a.kill()	
17	Animal.show_count()	

8-1-10　類別方法

- 第1 到9 行：定義類別Animal，初始化類別變數count 為0，變數count 在方法外部且前方沒有加上self
- 變數count 是類別變數，以此類別宣告的所有物件共用一個類別變數，類別變數count 使用「Animal.count」或「cls.count」進行存取。

- 定義方法「__init__」，方法「__init__」輸入參數為self，將類別Animal 的變數count 遞增1(第3 到4 行)。
- 定義方法「kill」，輸入參數為self，將類別Animal 的變數count 遞減1(第5 到6 行)。定義show_count 為類別方法，顯示字串「現在有」，串接cls.count，串接「隻動物」在螢幕上(第7 到9 行)。

- 第10 行：宣告一個類別Animal 的物件，變數a參考到此物件。
- 第11 行：呼叫類別Animal 的方法show_count。
- 第12 行：宣告一個類別Animal 的物件，變數b參考到此物件。

- 第13 行：呼叫類別Animal 的方法show_count。
- 第14 行：宣告一個類別Animal 的物件，變數c參考到此物件。
- 第15 行：呼叫類別Animal 的方法show_count。

- 第16 行：呼叫物件a 的方法kill。
- 第17 行：呼叫類別Animal 的方法show_count。

8-1-11　靜態方法

- 靜態方法(static method) 讓類別不需要建立物件，就可以直接使用該類別的靜態方法，需在類別中方法的前一行使用裝飾器「@staticmethod」。

行號	範例 (：ch8\8-1-11- 靜態方法 .py)	執行結果
1	class Say():	
2	@staticmethod	
3	def hello():	Hello
4	print('Hello')	
5	Say.hello()	

程式解說

- 第1到4行：定義類別Say，定義靜態方法hello，使用函式print顯示字串「Hello」在螢幕上。
- 第5 行：呼叫類別Animal 的靜態方法hello。

8-2 例外(exception)

- 在執行程式的過程中產生錯誤，程式會中斷執行，發出例外訊息，以下介紹例外的程式區塊，與實作自訂的例外類別。

8-2-1 try-except

- 使用程式區塊「try…except…」可以攔截例外，在try 區塊中撰寫可能發生錯誤的程式，若發生錯誤，則會跳到except 區塊執行進行後續的處理。

8-2-1 try-except

行號	範例 (：ch8\8-2-1-try-except.py)	執行結果
1 2 3 4	try: pwd = input(' 請輸入密碼 ') except: print(' 發生錯誤 ')	輸入數字與字元則不會發生錯誤，變數 pwd 會參考到此密碼。若輸入「ctrl+D」，則顯示「發生錯誤」。 (1) 輸入數字與字元 請輸入密碼 abc123 (2) 輸入「Ctrl+D」 請輸入密碼 ^D 發生錯誤

程式解說

- 第1 到4 行：顯示「請輸入密碼」在螢幕上，使用函式input 輸入密碼，變數pwd會參考到此密碼，若輸入過程中發生錯誤，例如：輸入「Ctrl+D」，則會跳到except 區塊去執行，顯示「發生錯誤」。

8-2-2 try-except-else

- 使用程式區塊「try…except…else…」可以攔截例外，在try 區塊中撰寫可能發生錯誤的程式，若發生錯誤，則會跳到except 區塊進行後續的處理，若沒有發生錯誤，則會跳到else 區塊執行。

8-2-2 try-except-else

- except 後可以接指定錯誤類型，常見錯誤類型，如下表。

表 8-4 常見的錯誤類型

錯誤類型	說明
KeyboardInterrupt	當使用者輸入中斷 (Ctrl+C) 時，發出此錯誤。
ZeroDivisionError	除以 0 時，發出此錯誤。
EOFError	接受到 EOF(end of file) 訊息時，發出此錯誤。
NameError	區域變數或全域變數找不到時，發出此錯誤。
OSError	與作業系統有關的錯誤
FileNotFoundError	檔案或資料夾找不到時，發出此錯誤。
ValueError	輸入資料與程式預期輸入資料型別不同時，發出此錯誤。

8-2-2 try-except-else

行號	範例 ( : ch8\8-2-2-try-except-else.py)	執行結果

1
2
3
4
5
6
	try:
	  pwd = input(' 請輸入密碼 ')
	except EOFError:
	  print(' 輸入 EOF')
	else:
	  print(' 輸入密碼爲 ', pwd)
		若輸入「Ctrl+D」，則發出 EOFError 錯誤，顯示「輸入 EOF」；若輸入數字與字元則不會發生錯誤，密碼會儲存在變數 pwd，接著執行 else 區塊顯示「輸入密碼爲」與變數 pwd。
		(1) 輸入數字與字元
		請輸入密碼 abc123
		輸入密碼爲 abc123
		(2) 輸入「Ctrl+D」
		請輸入密碼 ^D
		輸入 EOF

程式解說

- 第1 到6 行：顯示「請輸入密碼」在螢幕上，使用函式input 輸入密碼，變數pwd會參考到此密碼。(第1 到2 行)。

- 若輸入過程中輸入「ctrl+D」，則會跳到except EOFError 區塊去執行，顯示「輸入EOF」(第3 到4 行)
- 若沒有發生錯誤，則會跳到else 區塊去執行，顯示「輸入密碼為」與變數pwd 到螢幕上(第5 到6 行)。

8-2-3 try-except-as-else

- 使用程式區塊「try…except…as…else…」可以攔截例外，在try 區塊中撰寫可能發生錯誤的程式，若發生錯誤，則會跳到except區塊進行後續的處理

- 在except 後面接上as 就會將錯誤類別轉換成對應的錯誤類別物件，except 區塊個數可以有很多個，區分各種錯誤的類型，except 區塊內撰寫對應的錯誤處理程式；若沒有發生錯誤，則會跳到else 區塊執行。

8-2-3 try-except-as-else

行號	範例 ( : ch8\8-2-3-try-except-as-else.py)
1	try:
2	num = int(input(' 請輸入整數 '))
3	except EOFError:
4	print(' 輸入 EOF')
5	except ValueError as ve:
6	print(' 發生 ValueError 錯誤 ',ve)
7	except Exception as e:
8	print(' 發生其他錯誤 ',e)
9	else:
10	print(' 輸入整數爲 ', num)

8-2-3 try-except-as-else

- 第1 到10 行：顯示「請輸入整數」在螢幕上，使用函式input 輸入整數，使用函式int 將字串轉換成整數物件，變數num參考到此物件(第1 到2 行)。
- 若輸入過程中輸入「ctrl+D」，則會跳到except EOFError 區塊去執行，顯示「輸入EOF」(第3 到4行)

- 若輸入字串「abc」則會跳到except ValueError as ve 區塊去執行，顯示「發生 ValueError 錯誤」與物件ve 的訊息(第5 到 6 行)

- 若是其他的錯誤則會跳到except Exception as e 區塊去執行，顯示「發生其他錯誤」與物件e 的訊息(第7 到8 行)
- 若沒有發生錯誤，則會跳到else 區塊去執行，顯示「輸入整數為」與變數num 到螢幕上(第9 到10 行)。

執行結果

- 若輸入「Ctrl+D」，則發出EOFError 錯誤，顯示「輸入EOF」
- 若輸入英文字串則發出ValueError 錯誤，命名為ve，顯示「發生ValueError」與錯誤訊息ve

- 若其他錯誤，則發出Exception 錯誤，命名為e，顯示「發生其他錯誤」與錯誤訊息e
- 若沒有出現錯誤，變數num會參考到輸入的整數，接著執行else 區塊顯示「輸入整數為」與變數num。

(1) 輸入數字

```
請輸入整數 123
輸入整數爲 123
```

(2) 輸入「Ctrl+D」相當於輸入 EOF

```
請輸入整數 ^D
輸入 EOF
```

(3) 輸入「abc」

```
請輸入整數 abc
發生 ValueError 錯誤 invalid literal for int() with base 10: 號 abc 號
```

8-2-4 try-except-as-else 與自訂例外類別

- 可以自訂例外類別，自訂例外類別需要繼承系統例外類別Exception，該類別就會成為例外類別，可以傳入參數到自訂例外類別，將錯誤資訊儲存在自訂例外類別，使用指令raise 發出例外，接著由except 進行例外處理。

8-2-4 try-except-as-else 與自訂例外類別

行號	範例 (：ch8\8-2-4- 自訂例外類別 .py)
1	class PwdException(Exception):
2	def __init__(self,pwd,len):
3	super().__init__(self)
4	self.pwd=pwd
5	self.len=len
6	try:
7	pwd = input(' 請輸入密碼，長度至少 8 個字元 ')
8	if len(pwd) < 8:
9	raise PwdException(pwd,len(pwd))
10	except EOFError:
11	print(' 輸入 EOF')
12	except PwdException as pex:
13	print(' 密碼 ', pex.pwd, ' 長度爲 ', pex.len, ' 密碼長度不足 ')
14	else:
15	print(' 輸入密碼爲 ', pwd)

程式解說

- 第1 到5 行：定義類別PwdException 繼承類別Exception，定義方法「__init__」，方法「__init__」輸入參數為self、pwd 與len
- 使用「super()」呼叫基礎類別Exception 的方法「__init__」傳入self，設定self.pwd 為pwd，設定self.len 為len（第2 到5 行）。

- 第6 到15 行：顯示「請輸入密碼，長度至少8 個字元」在螢幕上，使用函式input輸入密碼，變數pwd參考到此密碼(第7 行)。
- 若pwd 的長度小於8，則發出自訂的例外類別PwdException，輸入參數pwd 與pwd 的長度(第8 到9 行)

- 若輸入過程中輸入「ctrl+D」，則會跳到except EOFError 區塊去執行，顯示「輸入EOF」(第10 到11 行)
- 若長度小於8 則跳到except PwdException as pex 區塊去執行，顯示「密碼」串接pex.pwd 串接「長度為」串接pex.len 串接「密碼長度不足」(第12 到13 行)

- 若沒有發生錯誤，則會跳到else 區塊去執行，顯示「輸入密碼為」與變數pwd 到螢幕上(第14 到15 行)。

執行結果

- 若輸入「Ctrl+D」，則發出EOFError 錯誤，顯示「輸入EOF」
- 若輸入密碼長度小於8，則發出PwdException 錯誤，顯示密碼、密碼長度與「密碼長度不足」
- 若沒有出現錯誤，密碼會儲存在變數pwd，接著執行else 區塊顯示「輸入密碼為」接著顯示變數pwd。
- 以下為三種執行結果。

(1) 輸入密碼長度大於等於 8 個字元

```
請輸入密碼，長度至少 8 個字元 abcd1234
輸入密碼爲 abcd1234
```

(2) 輸入「Ctrl+D」相當於輸入 EOF

```
請輸入密碼，長度至少 8 個字元 ^D
輸入 EOF
```

(3) 輸入「abc」

```
請輸入密碼，長度至少 8 個字元 abc
密碼 abc 長度爲 3 密碼長度不足
```

8-2-5 try-except-as-else-finally 與自訂例外類別

- 使用程式區塊「try…except…as…else…finally… 」可以攔截例外，在try 區塊中撰寫可能發生錯誤的程式

- 若發生錯誤，則會跳到except 區塊進行後續的處理，在except 後面接上as 就會將錯誤類別轉換成對應的錯誤類別物件，except 區塊個數可以有很多個，區分各種錯誤的類型，except 區塊內撰寫對應的錯誤處理程式

- 若沒有發生錯誤，則會跳到else 區塊執行，不管有沒有發生錯誤，最後都要執行finally 區塊。

8-2-5 try-except-as-else-finally 與自訂例外類別

行號	範例 (: ch8\8-2-5-try-except-as-else-finally.py)
1	class PwdException(Exception):
2	def __init__(self,pwd,len):
3	super().__init__(self)
4	self.pwd=pwd
5	self.len=len
6	try:
7	pwd = input(' 請輸入密碼，長度至少 8 個字元 ')
8	if len(pwd) < 8:
9	raise PwdException(pwd,len(pwd))
10	except EOFError:
11	print(' 輸入 EOF')
12	except PwdException as pex:
13	print(' 密碼 ', pex.pwd, ' 長度爲 ', pex.len, ' 密碼長度不足 ')
14	else:
15	print(' 輸入密碼爲 ', pwd)
16	finally:
17	print(' 請妥善保管密碼 ')

程式解說

- 第1 到5 行：定義類別PwdException 繼承類別Exception，定義方法「__init__」
- 方法「__init__」輸入參數為self、pwd 與len， 使用「super() 」呼叫基礎類別Exception 的方法「__init__」傳入self，設定self.pwd 為pwd
- 設定self.len 為len(第2 到5 行)。

- 第6 到17 行：顯示「請輸入密碼，長度至少8 個字元」在螢幕上，使用函式input輸入密碼，變數pwd參考到此密碼(第7 行)。
- 若pwd 的長度小於8，則發出自訂的例外類別PwdException，輸入參數pwd 與pwd 的長度(第8 到9 行)

- 若輸入過程中輸入「ctrl+D」，則會跳到except EOFError 區塊去執行，顯示「輸入EOF」(第10 到11 行)

- 若長度小於8，會跳到except PwdException as pex 區塊去執行，顯示「密碼」，串接pex.pwd，串接「長度為」，串接pex.len，串接「密碼長度不足」(第12 到13 行)

- 若沒有發生錯誤，則會跳到else 區塊去執行，顯示「輸入密碼為」與變數pwd 到螢幕上；最後不管有沒有錯誤，都會執行finally 區塊，顯示「請妥善保管密碼」。

執行結果

- 若輸入「Ctrl+D」，則發出EOFError 錯誤，顯示「輸入EOF」
- 若輸入密碼長度小於8，則發出PwdException 錯誤，顯示密碼、密碼長度與「密碼長度不足」
- 若沒有出現錯誤，變數pwd參考到此密碼，接著執行else 區塊顯示「輸入密碼為」與變數pwd，最後都會執行finally 區塊，顯示「請妥善保管密碼」。以下為三種執行結果。

(1) 輸入密碼長度大於等於 8 個字元

```
請輸入密碼，長度至少 8 個字元 abcd1234
輸入密碼爲 abcd1234
請妥善保管密碼
```

(2) 輸入「Ctrl+D」相當於輸入 EOF

```
請輸入密碼，長度至少 8 個字元 ^D
輸入 EOF
請妥善保管密碼
```

(3) 輸入「abc」

```
請輸入密碼，長度至少 8 個字元 abc
密碼 abc 長度爲 3 密碼長度不足
請妥善保管密碼
```